

LAPORAN PRAKTIKUM: SEGMENTASI CITRA REAL-WORLD
PENGOLAHAN CITRA DIGITAL



Dosen Pengampu:
Dr. Yasdinul Huda, S.Pd., M.T.

Oleh:
Muhammad Zahran
24343077

PROGRAM STUDI INFORMATIKA
DEPARTEMEN TEKNIK ELEKTRONIKA
FAKULTAS TEKNIK
UNIVERSITAS NEGERI PADANG
2026

A. Metodologi Segmentasi

Program ini menggunakan tiga pendekatan utama dalam melakukan segmentasi citra:

1. Thresholding-Based:

- *Global Thresholding*: Menggunakan nilai ambang statis ($T = 127$).
- *Otsu's Method*: Menghitung nilai ambang optimal secara otomatis berdasarkan distribusi histogram bimodal.
- *Adaptive Thresholding (Mean & Gaussian)*: Menentukan nilai ambang secara lokal berdasarkan area tetangga, berguna untuk menangani pencahayaan tidak rata.

2. Edge-Based (Deteksi Tepi):

- *Sobel & Prewitt*: Menggunakan operator gradien untuk mencari transisi intensitas.
- *Canny Edge Detection*: Algoritma deteksi tepi multi-tahap yang lebih kompleks (penghalusan, pencarian gradien, *non-maximum suppression*, dan *hysteresis thresholding*).

3. Region-Based:

- *Connected Components*: Mengelompokkan piksel terhubung setelah proses thresholding.
- *Watershed Transformation*: Metode berbasis morfologi untuk memisahkan objek yang saling bersentuhan menggunakan *distance transform*.

B. Tabel Perbandingan Metrik

Berdasarkan hasil eksekusi program, berikut adalah ringkasan performa metode terbaik untuk setiap karakteristik citra:

1. Citra Bimodal (kunci.jpg)

--- Evaluasi Citra: Bimodal (kunci.jpg) ---						
Metode	IoU	Dice	Acc	Prec	Rec	Waktu (s)
Global (T=127)	0.9861	0.9930	0.9875	0.9988	0.9873	0.000671
Otsu	0.9973	0.9986	0.9975	0.9976	0.9997	0.002148
Adaptive Mean	0.9168	0.9566	0.9195	0.9331	0.9813	0.004765
Adaptive Gaussian	0.9149	0.9556	0.9171	0.9262	0.9869	0.012663
Sobel	0.0036	0.0071	0.0857	0.1945	0.0036	0.056574
Prewitt	0.0038	0.0076	0.0860	0.2047	0.0039	0.009493
Canny	0.0013	0.0027	0.0879	0.1148	0.0013	0.006857
Connected Comp	0.9981	0.9990	0.9983	0.9991	0.9990	0.023910
Watershed	0.0004	0.0007	0.0317	0.0054	0.0004	0.032889

>> Metode Terbaik untuk Bimodal (berdasarkan IoU): Connected Comp (IoU: 0.9981)

2. Uneven Illumination (iluminasi.jpdp)

```
--- Evaluasi Citra: Uneven Illumination (iluminasi.jpg) ---
```

Metode	IoU	Dice	Acc	Prec	Rec	Waktu (s)
Global (T=127)	0.5665	0.7232	0.9110	0.9910	0.5694	0.000658
Otsu	0.9466	0.9726	0.9886	0.9529	0.9931	0.004526
Adaptive Mean	0.1963	0.3282	0.2693	0.2020	0.8734	0.007847
Adaptive Gaussian	0.1921	0.3223	0.2380	0.1969	0.8869	0.022849
Sobel	0.0613	0.1156	0.7800	0.3234	0.0704	0.071830
Prewitt	0.0571	0.1080	0.7843	0.3481	0.0639	0.007523
Canny	0.0241	0.0471	0.7881	0.2903	0.0257	0.008402
Connected Comp	0.9503	0.9745	0.9895	0.9707	0.9783	0.023519
Watershed	0.0286	0.0555	0.0846	0.0352	0.1317	0.112483

```
>> Metode Terbaik untuk Uneven Illumination (berdasarkan IoU): Connected Comp (IoU: 0.9503)
```

3. Overlapping (overlapping.jpg)

```
--- Evaluasi Citra: Overlapping (overlapping.jpg) ---
```

Metode	IoU	Dice	Acc	Prec	Rec	Waktu (s)
Global (T=127)	0.9095	0.9526	0.9446	0.9180	0.9899	0.000351
Otsu	0.9345	0.9661	0.9613	0.9515	0.9812	0.001161
Adaptive Mean	0.5291	0.6921	0.6313	0.6531	0.7360	0.003560
Adaptive Gaussian	0.5118	0.6771	0.5990	0.6192	0.7470	0.009435
Sobel	0.2347	0.3801	0.4696	0.5556	0.2889	0.032451
Prewitt	0.2258	0.3684	0.4855	0.5963	0.2665	0.004757
Canny	0.1768	0.3005	0.4924	0.6702	0.1936	0.011116
Connected Comp	0.8401	0.9131	0.9094	0.9920	0.8458	0.011471
Watershed	0.0005	0.0011	0.1837	0.0017	0.0008	0.036805

```
>> Metode Terbaik untuk Overlapping (berdasarkan IoU): Otsu (IoU: 0.9345)
```

C. Visualisasi Hasil

Program menghasilkan grid visualisasi yang membandingkan citra asli dengan hasil segmentasi dari berbagai algoritma.

1. **Citra Asli:** Menampilkan objek (kunci, botol, atau beras) dalam skala abu-abu.
2. **Ground Truth:** Menggunakan Pseudo-GT yang dihasilkan secara otomatis melalui Otsu yang dibersihkan dengan operasi morfologi.
3. **Hasil Segmentasi:** Visualisasi biner (hitam-putih) dari setiap metode.
4. **Overlay Kontur:** Prediksi terbaik (berdasarkan IoU tertinggi) ditampilkan kembali di atas citra asli dengan garis tepi merah untuk memvalidasi presisi deteksi objek.

D. Analisis dan Rekomendasi

1. Analisis Karakteristik

- Pada citra Bimodal, hampir semua metode berbasis thresholding (Otsu & Global) bekerja sangat sempurna karena kontras objek dan latar belakang sangat tegas.
- Pada **Uneven Illumination**, Global Thresholding gagal total di area bayangan, sedangkan Otsu masih mampu beradaptasi. Perlu dicatat bahwa skor tinggi pada Otsu/Connected Comp di sini terjadi karena Ground Truth-nya sendiri dibuat menggunakan basis Otsu (*Bias Pseudo-GT*).

- Pada Overlapping (beras), metode Watershed yang diimplementasikan mendapatkan skor IoU sangat rendah. Hal ini dikarenakan kontras butir beras dengan anyaman bambu sangat tipis, sehingga *distance transform* tidak mampu menciptakan *marker* yang akurat.

2. Rekomendasi

- Untuk citra dengan pencahayaan tidak merata, sebaiknya gunakan Adaptive Thresholding dengan tuning parameter block size yang lebih spesifik.
- Untuk objek tumpah tindih (beras), diperlukan tahap *Pre-processing* tambahan seperti *Contrast Limited Adaptive Histogram Equalization* (CLAHE) sebelum masuk ke algoritma Watershed.
- Gunakan Ground Truth manual (labeling tangan) untuk mendapatkan evaluasi metrik yang lebih objektif dan valid.

E. Lampiran Kode Python

1. Kode Python

```

1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import time
5 import os
6 import warnings
7 warnings.filterwarnings('ignore')
8
9 # =====
10 # PIPELINE SEGMENTASI CITRA REAL-WORLD - MAHMOUD ZHARAN (2024M77)
11 # =====
12
13 DATA_DIR = './'
14
15 # Nama file gambar input yang Anda berikan
16 input_filenames = [
17     'Rimada', 'Rimada.jpg',
18     'Uyuen', 'Uyuen.jpg',
19     'Overlapping', 'Overlapping.jpg'
20 ]
21
22 def load_data(data_dir, assignment_dir):
23     """Memuat citra real dan ground truth (opsional)"""
24     datasets = []
25
26     for characteristic, filename in assignment_dir.items():
27         # Memuat citra input dari folder grayscale
28         img_path = os.path.join(data_dir, filename)
29         if not os.path.exists(img_path):
30             raise FileNotFoundError(f"Citra input '{img_path}' tidak ditemukan. Silakan taruh di folder '{DATA_DIR}'.")
31         img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
32
33         # Memuat dan memuat ground truth yang sesuai
34         name_no_ext = os.path.splitext(filename)[0]
35         gt_filename = name_no_ext + "_gt.png"
36         gt_path = os.path.join(data_dir, gt_filename)
37
38         gt_bin = None
39         gt_source = "auto"
40         if os.path.exists(gt_path):
41             gt = cv2.imread(gt_path, cv2.IMREAD_GRAYSCALE)
42             gt_bin = cv2.threshold(gt, 0, 255, cv2.THRESH_BINARY)
43             gt_source = "file"
44         else:
45             gt_bin = generate_mask(img)
46
47         # Catatan: Citra untuk beras (Overlapping)
48         # Untuk tujuan evaluasi pixel-wise yang konsisten (IoU), user harus memuat
49         # mask binary di mana CITRA bujur-bujur (dan hanya beras) adalah patch.
50         # Di sini, saya sangat yakin dan memodifikasi pemrosesan manual yang benar.
51
52         datasets.append((characteristic, img, gt_bin, gt_source))
53
54     return datasets
55
56 def evaluate_metrics(gt, pred):
57     """Menghitung metrik evaluasi segmentasi binary"""
58     gt_bin = (gt > 127).astype(np.uint8).flatten()
59     pred_bin = (pred > 127).astype(np.uint8).flatten()
60
61     intersection = np.logical_and(gt_bin, pred_bin)
62     union = np.logical_or(gt_bin, pred_bin)
63
64     # Hitung pixel TRUE (Objek), FALSE (Background)
65     tp = np.sum(intersection)
66     fp = np.sum(pred_bin == 1 & (gt_bin == 0))
67     fn = np.sum(pred_bin == 0 & (gt_bin == 1))
68     tn = np.sum(pred_bin == 0 & (gt_bin == 0))
69
70     # Hitung metrik
71     tpr = tp / (tp + fp + fn + 1e-6)
72     dpr = 2 * tp / (2 * tp + fp + fn + 1e-6)
73     prc = tp / (tp + fp + 1e-6)
74     roc = tp / (tp + fn + 1e-6)
75
76     return tpr, dpr, prc, roc
77
78 def generate_mask(img):
79     """Membuat pseudo-ground-truth otomatis dari citra grayscale"""
80     # Opsi: thresholding cepat untuk segmentasi kasar
81     # 1. Opsi: thresholding cepat untuk segmentasi kasar
82     # 2. Opsi: thresholding cepat untuk segmentasi kasar
83     kernel = np.ones([5, 5], np.uint8)
84     gt_clean = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel, iterations=1)
85     gt_clean = cv2.morphologyEx(gt_clean, cv2.MORPH_CLOSE, kernel, iterations=1)
86     return gt_clean
87
88 # ===== METODE THRESHOLDING =====
89 def apply_thresholding(img):
90     results = {}
91
92     # Global Thresholding (Manual T=127)
93     start = time.time()
94     res_global = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
95     results['Global (T=127)'] = (res_global, time.time() - start)
96
97     # Otsu's Method
98     start = time.time()
99     res_otsu = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
100     results['Otsu'] = (res_otsu, time.time() - start)
101
102     # Adaptive Mean (Disarankan untuk Real Data)
103     start = time.time()
104     # Global parameterisasi size yang lebih besar (nilai 21 dan 5 yang lebih besar untuk real data)
105     res_adapt_mean = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY, 21, 5)
106     results['Adaptive Mean'] = (res_adapt_mean, time.time() - start)
107
108     # Adaptive Gaussian
109     start = time.time()
110     res_adapt_gauss = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 21, 5)
111     results['Adaptive Gaussian'] = (res_adapt_gauss, time.time() - start)
112
113     return results
114

```

```

114
115 # --- METHOD EDGE DETECTION ---
116 def apply_edges(img):
117     results = {}
118     img_binar = cv2.GaussianBlur(img, (3, 3), 0)
119
120     # Sobel
121     start = time.time()
122     sobelx = cv2.Sobel(img_binar, cv2.CV_64F, 1, 0, ksize=3)
123     sobely = cv2.Sobel(img_binar, cv2.CV_64F, 0, 1, ksize=3)
124     sobel_mag = np.sqrt(sobelx**2 + sobely**2)
125     sobel_res = np.uint8(255 * sobel_mag / np.max(sobel_mag))
126     # Threshold manual untuk binerisasi edge sobel
127     _, sobel_bin = cv2.threshold(sobel_res, 20, 255, cv2.THRESH_BINARY)
128     results['sobel'] = (sobel_bin, time.time() - start)
129
130     # Prewitt
131     start = time.time()
132     kernelx = np.array([[1, 1, 1], [0, 0, 0], [-1, -1, -1]])
133     kernely = np.array([[1, 0, -1], [-1, 0, 1], [-1, 0, 1]])
134     prewittx = cv2.filter2D(img_binar, -1, kernelx)
135     prewitty = cv2.filter2D(img_binar, -1, kernely)
136     prewitt_res = cv2.add(prewittx, prewitty)
137     # Threshold manual untuk binerisasi edge prewitt
138     _, prewitt_bin = cv2.threshold(prewitt_res, 30, 255, cv2.THRESH_BINARY)
139     results['prewitt'] = (prewitt_bin, time.time() - start)
140
141     # Canny (Disesuaikan untuk Real Data)
142     start = time.time()
143     # Threshold lebih rendah untuk menangkap tepi yang lebih lembut pada real data
144     canny_res = cv2.Canny(img_binar, 10, 100)
145     results['canny'] = (canny_res, time.time() - start)
146
147     return results
148
149 # --- METHOD REGION-BASED ---
150 def apply_region(img):
151     results = {}
152
153     # Connected Components (menggunakan hasil Otsu sebagai input)
154     start = time.time()
155     _, thresh = cv2.threshold(img, 0, 255, cv2.THRESH_OTSU + cv2.THRESH_OTSU)
156     res_labels, labels = cv2.connectedComponents(thresh)
157     cc_res = np.zeros_like(img)
158     if res_labels > 1:
159         # Menentukan label terbesar (summi objek utama, untuk kelas ini akan salah)
160         # Akan memilih objek lebih lama untuk kelas
161         largest_label_id = np.argmax([cv2.sumAreaLabel(labels, flattened(i)) for i in range(1, res_labels)]) + 1
162         cc_res[labels == largest_label_id] = 255
163         results['Connected Comp'] = (cc_res, time.time() - start)
164
165     # Watershed (Marker-based) - Disesuaikan untuk kelas
166     start = time.time()
167     img_color = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
168
169     # Pre-processing untuk kelas: Thresholding awal yang sangat sensitif
170     # Ini adalah bottleneck pada foto kelas karena kontras rendah.
171     _, thresh_low = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY_INV)
172
173     kernel = np.ones((3,3), np.uint8)
174     opening = cv2.morphologyEx(thresh_low, cv2.MORPH_OPEN, kernel, iterations=1)
175     sure_bg = cv2.dilate(opening, kernel, iterations=2)
176     dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
177
178     # Threshold watershed: KEDURUSAN threshold untuk menangkap kelas yang tumpang tindih
179     # Namun, karena kontras rendah dengan latar, ini mungkin akan memunculkan banyak noise.
180     _, sure_fg = cv2.threshold(dist_transform, 0.4 * dist_transform.max(), 255, 0)
181     sure_fg = cv2.morphologyEx(sure_fg, cv2.MORPH_OPEN, kernel)
182     unknown = cv2.subtract(sure_bg, sure_fg)
183     _, markers = cv2.connectedComponents(sure_fg)
184     markers[unknown == 255] = 0
185     markers = cv2.subtract(img_color, markers)
186     w_res = np.zeros_like(img)
187     w_res[markers > 0] = 255
188     results['watershed'] = (w_res, time.time() - start)
189
190     return results
191
192 def main():
193     print("=====")
194     print("PIPEDLINE SEGMENTASI CITRA REAL-WORLD - MUHAMMAD ZAHWAN (24343077)")
195     print("=====")
196     print("Data Dir: " + os.path.abspath(DATA_DIR))
197
198     # Memuat data
199     datasets = load_data(DATA_DIR, image_assignments)
200
201     try:
202         except FileNotFoundError as e:
203             print("Error: " + str(e))
204             return
205
206     all_methods = ["Global (T127)", "Otsu", "Adaptive Mean", "Adaptive Gaussian",
207                   "Sobel", "Prewitt", "Canny", "Connected Comp", "Watershed"]
208
209     # Konfigurasi Grid visualisasi layout
210     n_rows = 4
211     n_cols = 3
212
213     for name, img, gt, gt_source in datasets:
214         print(f"--- Dataset Citra: {name} ({image_assignments[name]}) ---")
215
216         # Threshold Manual
217         res_thresh = apply_thresholding(img)
218         res_edges = apply_edges(img)
219         res_region = apply_region(img)
220
221         combined_results = ("res_thresh", "res_edges", "res_region")
222
223         # Citra Hasil
224         if gt_source == "water":
225             print(f"Water: {('T127':id) | ('Otsu':id) | ('Sobel':id) | ('Prewitt':id) | ('Canny':id) | ('Watershed':id)}")
226             print("-" * 80)
227
228             best_idx = -1
229             best_method = ""
230
231             for method_name in all_methods:
232                 if method_name not in combined_results: continue
233                 pred, t = combined_results[method_name]
234                 f1, dice, acc, prec, rec = evaluate_metrics(gt, pred)
235                 print(f"{method_name} (t: {t:.4f}) | (acc: {acc}) | (dice: {dice}) | (prec: {prec}) | (rec: {rec}) | (f1: {f1})")
236                 if f1 > best_idx:
237                     best_idx = f1
238                     best_method = method_name
239
240             print(f"Metode Terbaik untuk {name} (berdasarkan F1): {best_method} (F1: {best_idx:.4f})")
241
242     # Visualisasi (Grid Comprehensive per image)
243     plt.figure(figsize=(20, 15))
244     plt.suptitle(f"Hasil Segmentasi Comprehensive - {name} ({image_assignments[name]}) by: M. Zahwan (24343077)", fontsize=18)
245
246     # Plot Original
247     plt.subplot(n_rows, n_cols, 1)
248     plt.imshow(img, cmap='gray')
249     plt.title('Citra Asli (Grayscale)', fontsize=14)
250     plt.axis('off')
251
252     # Plot Ground Truth
253     plt.subplot(n_rows, n_cols, 2)
254     plt.imshow(gt, cmap='gray')
255     if gt_source == "water":
256         plt.title('Ground Truth (Auto / Pseudo)', fontsize=14)
257     else:
258         plt.title('Ground Truth (Binary Mask)', fontsize=14)
259     plt.axis('off')
260
261     # Plot 3 segmentation results in a grid (from 3 to 11)
262     for i, method_name in enumerate(all_methods):
263         if method_name not in combined_results: continue
264         pred, t = combined_results[method_name]
265         plt.subplot(n_rows, n_cols, 3 + i)
266         plt.imshow(pred, cmap='gray')
267         plt.title(f'{method_name} (t: {t:.4f}) (F1: {evaluate_metrics(gt, pred)[0]:.4f})', fontsize=12)
268         plt.axis('off')
269
270     # Plot Best Prediction Overlay Container (di sel 12)
271     best_pred = combined_results[best_method][0]
272     plt.subplot(n_rows, n_cols, 12)
273
274     # Menarik kontur pada prediksi terbaik
275     contours = cv2.findContours(best_pred, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
276     img_color = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
277     cv2.drawContours(img_color, contours, -1, (255, 0, 0), 2) # Kontur Merah
278
279     # Visualisasi kontur dengan MP
280     plt.imshow(img_color)
281     plt.title(f"Visual Terbaik: Overlay Kontur Merah by: M. Zahwan (24343077)", fontsize=12, color='white', fontstyle='bold', backgroundcolor='red')
282     plt.axis('off')
283
284     # Plot layout
285     plt.tight_layout()
286     plt.subplots_adjust(top=0.92) # Tambahan spasi untuk subplot
287     plt.show()
288
289 if __name__ == "__main__":
290     main()

```

2. Screenshot Output

